

University of Macedonia
Department of Applied Informatics

2nd Laboratory Assignment

Neural Networks in Energy Disaggregation

Course: Neural Networks

Academic Year: 2025 – 2026

Student Name: Apostolos Sinioris

Student ID: ics24123

Supervisor: Prof. Eftychios Protopapadakis

July 1, 2026

Contents

1	Introduction	3
2	Dataset Description	4
2.1	Data Characteristics and Preprocessing	4
2.2	Data Visualization	5
3	Proposed Methodology	7
3.1	Model Architectures	7
3.1.1	Recurrent Neural Network (RNN)	7
3.1.2	Long Short-Term Memory (LSTM)	9
3.1.3	Gated Recurrent Unit (GRU)	11
3.2	Sequence-to-Sequence (Seq2Seq) Architecture	12
4	Training Process and Evaluation	14
4.1	Data Splitting Strategy	14
4.2	Hyperparameter Configuration	15
4.3	Early Stopping Mechanism	15
4.4	Evaluation Metrics	16
5	Experimental Results	17
5.1	Training Convergence	17
5.2	Error Metrics Analysis	19
5.3	Visual Performance	21
6	Conclusions	24

List of Figures

1	Random sequence sample 1 (Index 11387). Both aggregated and appliance consumption show low-level standby noise.	5
2	Random sequence sample 2 (Index 31640). The appliance is in a standby state with consumption around 0.23 Watts.	6
3	Random sequence sample 3 (Index 47490). Both household and appliance consumption are completely zero.	6
4	A simple recurrent network. At each time step t , activation is passed along solid edges as in a feedforward network. Dashed edges connect a source node at each time t to a target node at each following time $t + 1$	8
5	A visualization of the vanishing gradient problem. If the weight along the recurrent edge is less than one, the contribution of the input at the first time step to the output at the final time step will decrease exponentially fast as a function of the length of the time interval in between.	8
6	Detailed architecture of an LSTM memory cell. The diagram highlights the uninterrupted flow of the cell state $c(t)$ at the top, and the sequential operation of the Forget, Input, and Output gates regulating the hidden state $h(t)$ based on the current input $x(t)$	10
7	The internal architecture of a Gated Recurrent Unit (GRU) block, highlighting the streamlined design featuring only an update gate and a reset gate.	11
8	A general representation of a Sequence-to-Sequence (Encoder-Decoder) model. The encoder summarizes the input sequence into a context vector, which is then utilized by the decoder to generate the output sequence.	13
9	Training and validation loss (MSE) over epochs for the Simple RNN model.	17
10	Training and validation loss (MSE) over epochs for the LSTM model. Notice the large validation spike around epoch 10.	18
11	Training and validation loss (MSE) over epochs for the GRU model.	18
12	RMSE calculated per sequence in the test set.	19
13	MAE calculated per sequence in the test set.	19
14	Maximum Error calculated per sequence in the test set.	20
15	Appliance OFF state (Sequence 3399). Comparison of model predictions against the ground truth.	21
16	Appliance OFF state (Sequence 3028). Comparison of model predictions against the ground truth.	21
17	Appliance ON state (Sequence 316). The ground truth shows a sudden power spike.	22
18	Appliance ON state (Sequence 1352). The Simple RNN successfully detects the activation.	22

1 Introduction

This project aims to describe the application of Neural Network Architectures to **Non-intrusive load monitoring (NILM)**, known as **Energy Disaggregation**.

The recent energy crisis and rising electricity prices have led many households to look for ways of saving energy and reducing costs. Smart sensors and smart meters are a way of tracking daily power consumption in real-time, providing valuable data to consumers.

However, these devices usually only measure the total (aggregated) electricity used by the entire house. To truly understand the energy footprint and make targeted, energy-saving decisions, we need to know exactly how much power each individual appliance consumes.

Since installing a separate sensor on every single plug is expensive and impractical, a computational approach is required. This is where Non-Intrusive Load Monitoring (NILM), or energy disaggregation, becomes relevant.

NILM is a process for analyzing changes in the voltage and current going into a house and deducing what appliances are used in the house as well as their individual energy consumption. [1]

In this project, we are going to evaluate and compare the performance of 3 fundamental architectures used in Recurrent Neural Networks:

- A simple Recurrent Neural Network (**RNN**)
- A Long Short-Term Memory Neural Network (**LSTM**)
- A Gated Recurrent Unit Neural Network (**GRU**)

More specifically, these models will be trained on a provided dataset containing the aggregated power consumption of a household, with the goal of extracting the individual power waveform of a coffee machine.

Formulating this task as a **Sequence-to-Sequence (Seq2Seq) regression problem**, the models will predict the appliance's consumption for every given timestep.

Through training, hyperparameter tuning (e.g., Early Stopping), and evaluation using standard metrics (RMSE, MAE, Max Error), this study aims to identify the most suitable neural network architecture for modeling simple ON/OFF appliances characterized by abrupt power spikes.

2 Dataset Description

In this section, the provided dataset which is used to train and test the model, will be described.

We used Python as the programming language and Google Colab as the platform to test and run the model.

The Coffee Machine dataset

The dataset utilized in this project aims to provide the necessary sequence data to train models for identifying the power consumption of a coffee machine from the aggregated household load. The provided dataset consists of four text (.txt) files:

2.1 Data Characteristics and Preprocessing

- **Input_Data.txt:** Contains 50,000 rows (sequences) and 40 columns (timesteps/minutes). Each value represents the total (aggregated) power consumption of the house in Watts for a specific minute.
- **Output_Data.txt:** Has the exact same dimensions ($50,000 \times 40$). Each value represents the specific power consumption of the target appliance (Coffee Machine) in Watts for the corresponding minute.
- **CoffeeMachinemaxAgg.txt:** Contains the maximum aggregated consumption value ever recorded in the electrical panel. This is used to normalize the input values.
- **CoffeeMachinemaxApp.txt:** Contains the maximum consumption value recorded specifically for the coffee machine, used to normalize the output values.

The dataset was imported into the Google Colab platform, manually with code and without the help of external libraries.

```
1 # Load the dataset files using numpy's loadtxt function
2 # This converts the text files into numpy arrays
3 input_data = np.loadtxt(folder_path + 'Input_Data.txt', delimiter=',',
4                          ',')
5 output_data = np.loadtxt(folder_path + 'Output_Data.txt', delimiter=
6                          ',')
7
8 # Load the single values for maximum aggregated and appliance
9 consumption
10 max_agg = np.loadtxt(folder_path + 'CoffeeMachinemaxAgg.txt')
11 max_app = np.loadtxt(folder_path + 'CoffeeMachinemaxApp.txt')
```

Listing 1: Importing the data

Normalization is a crucial step for the models to work in a specific numerical range.

```
1 # 1. Normalization
2 # Divide input and output data by their respective maximum values
3 # This scales all values to be between 0 and 1
4 X_normalized = input_data / max_agg
5 y_normalized = output_data / max_app
```

Listing 2: Normalization of the data

RNNs require data in a specific shape (Samples, Timesteps, Features), so we add another one dimension, since we have 1 feature (Power) per timestep.

```

1  # 2. Reshape the data for RNNs (Samples, Timesteps, Features)
2  # We add a 3rd dimension since we have 1 feature (Power) per
   timestep
3  X_normalized = np.expand_dims(X_normalized, axis=-1)
4  y_normalized = np.expand_dims(y_normalized, axis=-1)

```

Listing 3: Adding a 3rd dimension

2.2 Data Visualization

In this section, we will represent some plots that visualize the data.

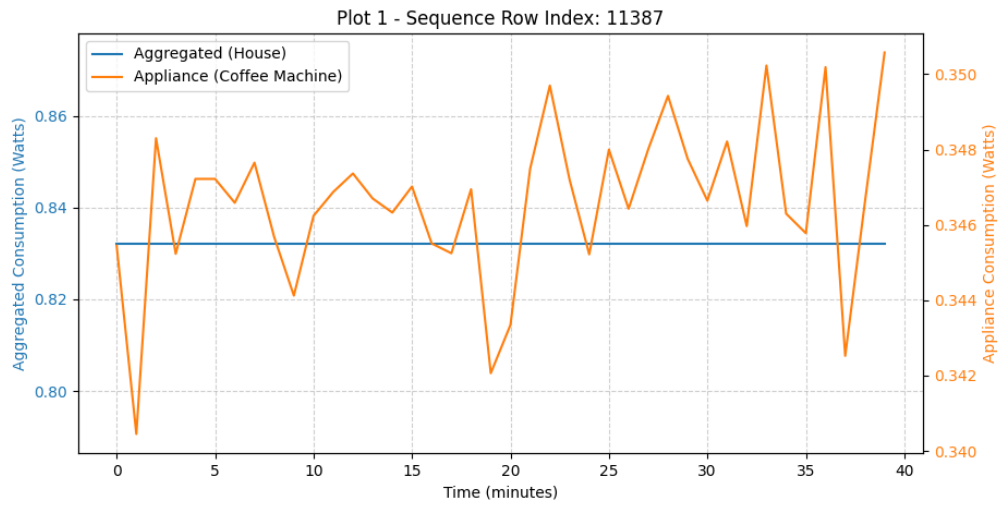


Figure 1: Random sequence sample 1 (Index 11387). Both aggregated and appliance consumption show low-level standby noise.

As seen in **Figure 1** (Sequence 11387), the blue line representing the total household consumption is completely flat at a very low level (around 0.83 Watts). At the same time, the orange line representing the coffee machine shows tiny fluctuations between 0.34 and 0.35 Watts. Because these values are so small, this most likely represents a "standby" state, where the machine is plugged in but not actively making coffee.

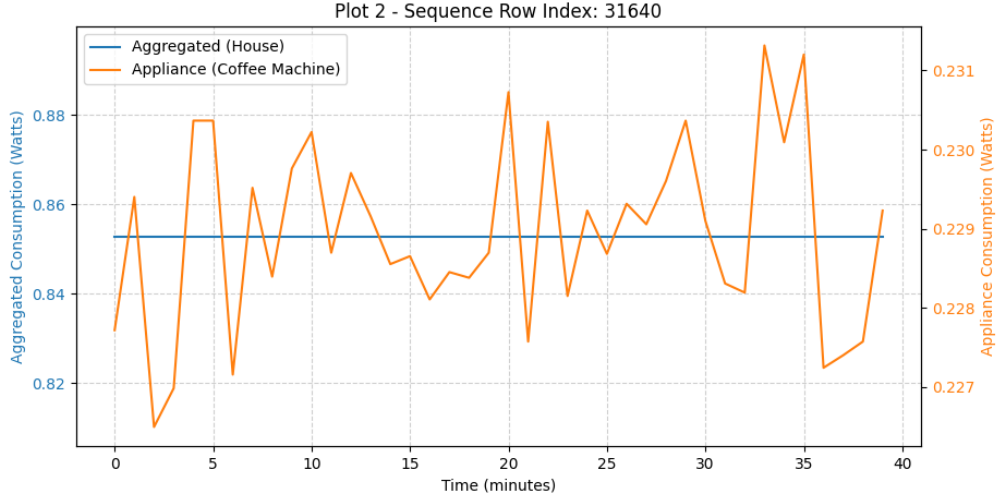


Figure 2: Random sequence sample 2 (Index 31640). The appliance is in a standby state with consumption around 0.23 Watts.

Figure 2 (Sequence 31640) shows a very similar situation. The household baseline is steady at 0.85 Watts, while the coffee machine has minor variations around 0.23 Watts. Again, this indicates that the appliance is not in active use, and these small changes are just electrical noise or standby power.

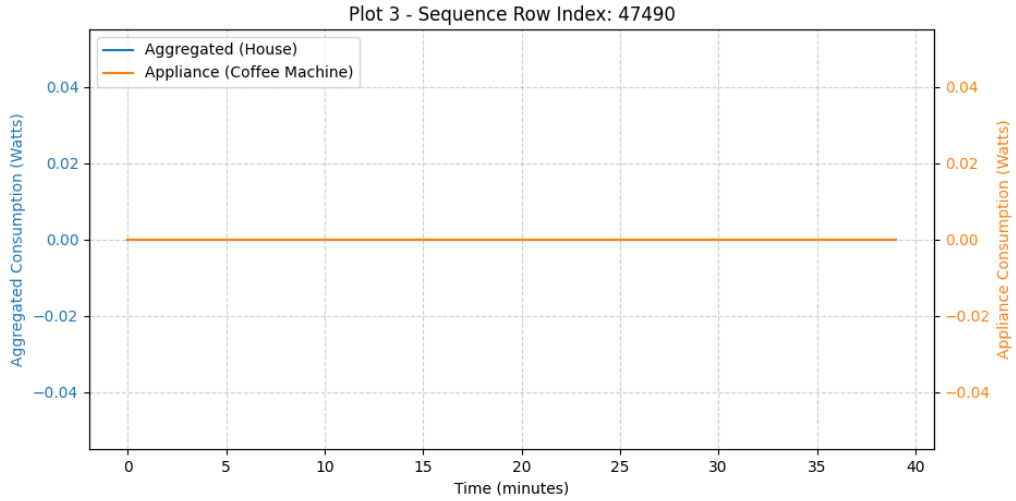


Figure 3: Random sequence sample 3 (Index 47490). Both household and appliance consumption are completely zero.

Finally, in **Figure 3** (Sequence 47490), both the total household consumption and the appliance consumption are exactly zero for the entire 40 minutes. This clearly represents a period where the appliance is completely turned off and the house draws no power.

3 Proposed Methodology

In this section, we will present all the methodologies that were used to build the model.

Terms like **RNN**, **LSTM** and **GRU** will be analysed, in order to understand their characteristics and see how they make predictions based on their training data.

3.1 Model Architectures

3.1.1 Recurrent Neural Network (RNN)

Recurrent Neural Networks (RNNs) are a type of Neural Network architecture designed to process sequential data, such as text, audio, and time-series. Unlike traditional feedforward networks, they are able to retain a state that can represent information from a long context window. Their structure consists of high-dimensional hidden states with non-linear dynamics.

The structure of the hidden states works as the memory of the network, and the state of the hidden layer at any given time is conditioned on its previous state. This structure enables the RNNs to store, remember, and process past complex signals for long time periods.

Furthermore, RNNs can map an input sequence to an output sequence at the current timestep and predict the sequence in the next timestep. [2]

A simple RNN has three layers. The **Input**, the **Recurrent Hidden**, and the **Output** layers. In Figure 4, information flows not only forward but also across time steps via the recurrent hidden state.

- **The Input Layer:** This layer takes a sequence of data vectors through time t , denoted as x_t , representing the features at the current timestep.
- **The Recurrent Hidden Layer:** This is the core of the network. At time t , it processes the current input x_t alongside the hidden state from the previous timestep h_{t-1} . By applying a non-linear activation function (such as $\tanh$), it generates the new hidden state h_t , which effectively acts as the network's updated memory.
- **The Output Layer:** This layer uses the current hidden state h_t to produce the final prediction y_t for that specific timestep.

The Vanishing Gradient Problem

Despite their theoretical capability to capture long-term dependencies, standard RNNs struggle in practice when dealing with very long sequences due to the **Vanishing Gradient Problem**. During the training process, RNNs use an algorithm called **Backpropagation Through Time (BPTT)**. In BPTT, the error (loss) is calculated at the output and the gradients are propagated backwards across all previous timesteps to update the network's weights.

However, as these gradients travel back through time, they are repeatedly multiplied by the network's weight matrices. If these values are small (less than 1), the gradients shrink exponentially with every step back. Eventually, the gradients approach zero and "vanish". As a result, the network fails to properly update its weights for the earliest inputs,

essentially "forgetting" old but potentially crucial information. A visual representation of this exponential decay over time can be seen in Figure 5 [3]

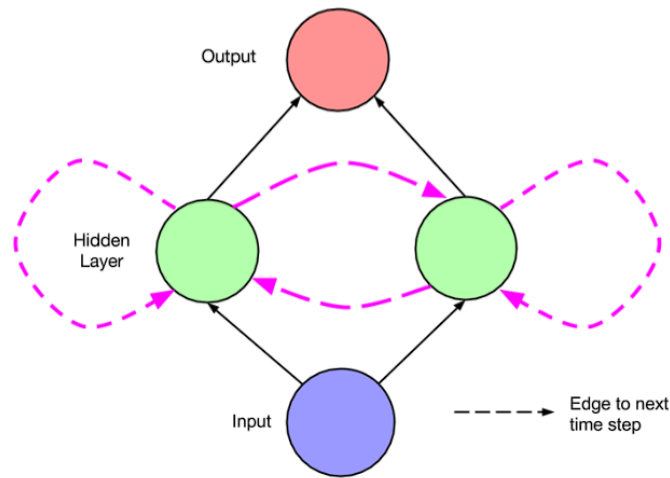


Figure 4: A simple recurrent network. At each time step t , activation is passed along solid edges as in a feedforward network. Dashed edges connect a source node at each time t to a target node at each following time $t + 1$.

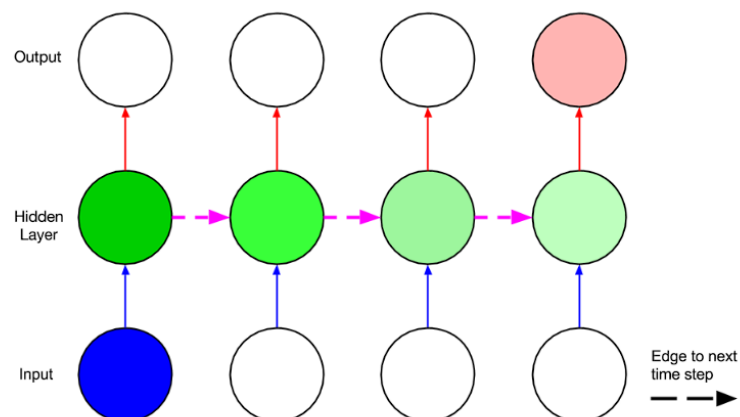


Figure 5: A visualization of the vanishing gradient problem. If the weight along the recurrent edge is less than one, the contribution of the input at the first time step to the output at the final time step will decrease exponentially fast as a function of the length of the time interval in between.

Figures 4 and 5, adapted from [3].

The Code Implementation

The implementation of the RNN model will be presented here as blocks of code.

```

1  # =====
2  # 1. Build Simple RNN Model
3  # =====
4  model_rnn = Sequential(name="RNN_Model")
5  model_rnn.add(SimpleRNN(64, input_shape=(seq_length, input_dim),
    return_sequences=True))

```

```

6      # The Dense layer outputs the final prediction for each timestep
7      model_rnn.add(Dense(output_dim))
8      model_rnn.compile(optimizer=Adam(learning_rate=0.001), loss=
      MeanSquaredError())

```

Listing 4: A simple implementation of the RNN model in Python

The implementation of the RNN model is constructed using the Keras `Sequential` API, which allows for building models layer by layer. The `SimpleRNN` is the Recurrent Hidden Layer, which takes as parameters the number of neurons (64), the input shape and a logical condition called `return_sequences` which is crucial for the model to predict every timestep (for every minute), instead of returning a final value.

3.1.2 Long Short-Term Memory (LSTM)

Long Short-Term Memory networks were introduced by Hochreiter, S., & Schmidhuber, J. (1997) [4] to offer a solution for the **Vanishing Gradient Problem**, mentioned in section 3.1.1.

Introduction

LSTMs were designed to overcome and reduce the effects of vanishing and exploding gradients. In standard architectures, error signals flowing backwards in time tend to either blow up or vanish, leading to an insufficient, decaying error backflow. By effectively addressing this issue, LSTMs are capable of storing information over extended time intervals, allowing them to successfully learn and maintain long-term dependencies in sequential data.

The Memory Cell

LSTMs resemble simple **RNNs** with the main difference, being that inside the hidden layers, each node is replaced by a *Memory Cell*. [3] A Memory Cell is a specialized building block, designed to process data across long sequences. Due to this building block, LSTMs can keep their hidden state through time and bypass the *Vanishing Gradient Problem*.

The Cell State acts as the core of this architecture. It resembles a conveyor belt that runs straight down the entire sequential chain. Instead of completely overwriting its content at every timestep like a standard RNN, the cell state allows information to flow along it with only minor linear interactions. This additive design ensures that long-term memory is smoothly carried forward, allowing error gradients to backpropagate without vanishing. [3]

Gates

The Cell State is controlled by a mechanism called *Gates*. Gates are regulating structures that control information flowing in and out of the cell state. A standard LSTM unit consists of 3 primary gates:

- **The Forget Gate:** This gate evaluates the current input and the previous hidden state to decide what information is no longer relevant and should be discarded from the cell state.
- **The Input Gate:** This gate determines which parts of the newly arrived information are important enough to be stored in the cell state.

- **The Output Gate:** This gate decides what specific information will be outputted (as the new hidden state) based on the current, updated cell state.

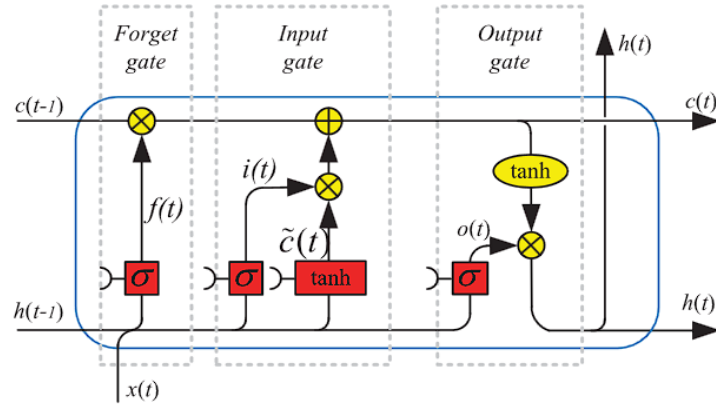


Figure 6: Detailed architecture of an LSTM memory cell. The diagram highlights the uninterrupted flow of the cell state $c(t)$ at the top, and the sequential operation of the Forget, Input, and Output gates regulating the hidden state $h(t)$ based on the current input $x(t)$.

In Figure 6, the internal architecture of the LSTM block uses these interconnected gates to carefully regulate the information. The horizontal line running across the top represents the **cell state** ($c(t)$), acting like a conveyor belt where information flows with only minor linear interactions.

The gates utilize sigmoid (σ) and hyperbolic tangent ($\tanh$) activation functions to squash values between 0 and 1, or -1 and 1, respectively. Through pointwise multiplication ($\otimes$) (or Hadamard multiplication) and pointwise addition ($\oplus$), the network precisely calculates what existing memory to forget, what candidate data to add, and what the final hidden state ($h(t)$) should be. Figure 6 adapted from [5].

The Code Implementation

The implementation of the LSTM model follows the same Seq2Seq principles as the simple RNN, utilizing the LSTM layer provided by the Keras library.

```

1  # =====
2  # 2. Build LSTM Model
3  # =====
4  model_lstm = Sequential(name="LSTM_Model")
5  model_lstm.add(LSTM(64, input_shape=(seq_length, input_dim),
6  return_sequences=True))
7  model_lstm.add(Dense(output_dim))
8  model_lstm.compile(optimizer=Adam(learning_rate=0.001), loss=
  MeanSquaredError())

```

Listing 5: Implementation of the LSTM model in Python

The core of the architecture is the LSTM layer, configured with 64 hidden units. Similar to the Simple RNN implementation, the `return_sequences=True` argument is explicitly defined. This is a critical requirement, as it forces the LSTM layer to output a sequence of predictions for every single timestep (each minute of the time window) rather than

returning just a single aggregated value at the very end. The hyperparameters of this model are kept identical to the Simple RNN architecture.

3.1.3 Gated Recurrent Unit (GRU)

Gated Recurrent Units (GRUs) were proposed by Cho et al. [6] to make each recurrent unit to adaptively capture dependencies of different time scales. Similarly to the LSTM unit, the GRU has gating units that modulate the flow of information inside the unit, however, without having a separate memory cell. [7]

Architectural Simplification

In contrast to the LSTM architecture, the GRU does not maintain a separate cell state (c_t). Instead, it merges the internal memory and the output into a single unified **hidden state** (h_t). This hidden state is entirely responsible for carrying the sequential information forward. By eliminating the separate memory cell and reducing the number of gates, the GRU significantly decreases the number of trainable parameters. This streamlined design makes the GRU computationally lighter and generally faster to train, while still effectively combating the *Vanishing Gradient Problem* [5, 7].

The Gating Mechanism

To control the flow of information, the GRU simplifies the traditional gating mechanism by utilizing only two distinct gates [5, 6]:

- **The Update Gate:** This gate essentially combines the roles of the forget and input gates found in LSTMs. It adaptively decides how much of the past hidden state should be retained and how much of the newly generated candidate information should be added to the final state.
- **The Reset Gate:** This gate determines how much of the previous hidden state should be ignored (or "forgotten") when computing the new candidate hidden state. If the reset gate evaluates to zero, the network completely drops the past information, acting as if it is reading the first symbol of a new sequence.

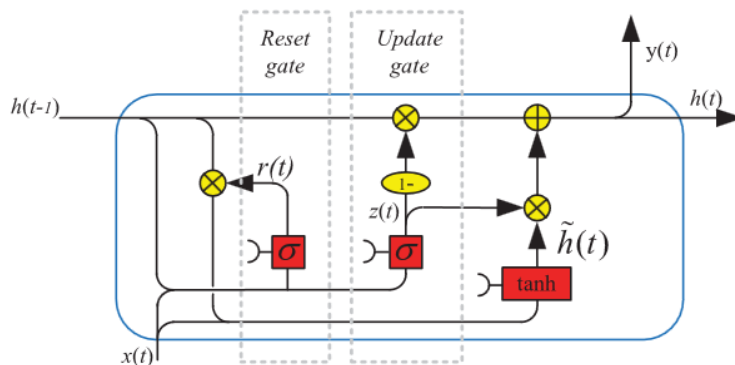


Figure 7: The internal architecture of a Gated Recurrent Unit (GRU) block, highlighting the streamlined design featuring only an update gate and a reset gate.

As depicted in Figure 7, the internal architecture of the GRU is noticeably less complex than the LSTM. Without the horizontal cell state line at the top, the information flow is

directly modulated by the update and reset gates within the main hidden state pathway. Figure adapted from [5].

The Code Implementation

```
1  # =====
2  # 3. Build GRU Model
3  # =====
4  model_gru = Sequential(name="GRU_Model")
5  model_gru.add(GRU(64, input_shape=(seq_length, input_dim),
6  return_sequences=True))
7  model_gru.add(Dense(output_dim))
8  model_gru.compile(optimizer=Adam(learning_rate=0.001), loss=
9  MeanSquaredError())
```

Listing 6: Implementation of the GRU model in Python

The GRU model is configured identically to both the Simple RNN and the LSTM. This allows any performance differences observed during evaluation to be safely attributed to the design of the specific recurrent cell (Simple, LSTM, or GRU).

3.2 Sequence-to-Sequence (Seq2Seq) Architecture

Traditional deep neural networks operate under the assumption that inputs and targets possess a fixed dimensionality. This structural constraint severely limits their applicability to time-series problems, where the length of the data sequences can vary or where temporal context is crucial.

To overcome this limitation, the **Sequence-to-Sequence (Seq2Seq)** architecture was introduced. A Seq2Seq model is specifically designed to map an input sequence to a corresponding output sequence, making it highly effective for temporal data [8].

In its fundamental form, the Seq2Seq architecture consists of two interacting recurrent neural networks: an *Encoder* and a *Decoder*. The Encoder network sequentially processes the input data, one timestep at a time. As it reads the input sequence, its hidden state changes, accumulating information.

As illustrated in Figure 8, once the entire input sequence is processed, the final hidden state of the encoder acts as a summary. It forms a fixed-length context vector that captures the entire input sequence and serves as the sole informational bridge between the two networks [6].

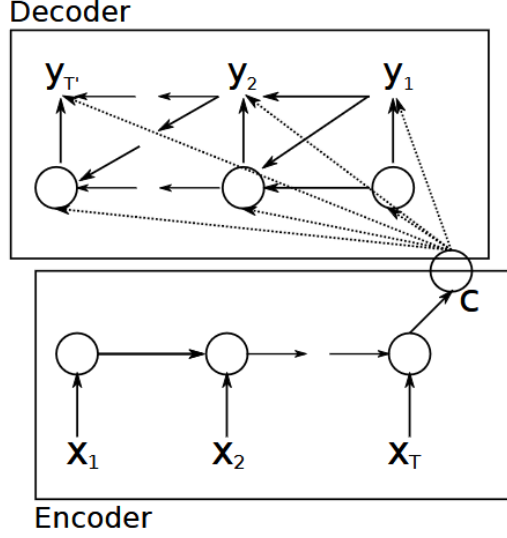


Figure 8: A general representation of a Sequence-to-Sequence (Encoder-Decoder) model. The encoder summarizes the input sequence into a context vector, which is then utilized by the decoder to generate the output sequence.

Figure 8 adapted from [6].

Following the encoding phase, the decoder network takes over. The decoder is essentially another recurrent neural network that is initialized with the context vector generated by the encoder. Its purpose is to unfold this summarized representation and extract the target output sequence, generating predictions step-by-step until the complete output sequence is formed [8, 6].

Application to Energy Disaggregation

While originally developed for natural language processing tasks such as machine translation, the Seq2Seq approach has proven to be highly effective in the context of Non-Intrusive Load Monitoring (NILM).

In energy disaggregation, instead of translating words from one language to another, the neural network is trained to translate an electrical signal. Specifically, it maps an input sequence consisting of the aggregated household power readings to an output sequence representing the individual power consumption of a target appliance [9].

This sequence-based approach is critical for electrical loads because the consumption of an appliance (like a coffee machine) is not a single isolated point, but a temporal signature consisting of activation spikes and steady states. By extracting a temporal window of aggregate power, the model utilizes the surrounding context to accurately output a window of appliance power [9].

In our specific project implementation, the architecture simplifies this concept by matching the input and output dimensions. By configuring the recurrent layers with the `return_sequences=True` argument, the network directly predicts the appliance's continuous electrical consumption (in Watts) for every individual timestep across the evaluated m -minute window.

4 Training Process and Evaluation

This section details the experimental setup utilized to train and evaluate the recurrent neural network models. Specifically, it outlines the data splitting strategy, the chosen hyperparameter configuration, the implementation of the early stopping mechanism, and the regression metrics used to evaluate the models' performance in accurately predicting the appliance's power consumption.

4.1 Data Splitting Strategy

The dataset was partitioned into three subsets using the **Holdout method**. Data sequences were randomly shuffled to eliminate any inherent ordering or temporal bias in the dataset. A fixed random seed (`seed = 42`) was utilized during this process to guarantee the exact reproducibility of the experiments. The data was ultimately divided into a **70% Training set**, a **15% Validation set**, and a **15% Testing set**.

The Code Implementation

```
1  # 3. Create indices and shuffle them randomly
2  total_samples = X_normalized.shape[0]
3  indices = np.arange(total_samples)
4
5  # Set a random seed so we get the exact same random split every time
6  # we run the code
7  np.random.seed(42)
8  np.random.shuffle(indices)
9
10 # 4. Define split percentages: 70% Train, 15% Validation, 15% Test
11 train_split = int(0.7 * total_samples)
12 val_split = int(0.85 * total_samples) # 70% + 15% = 85%
13
14 # Generate the specific indices for each set
15 train_idx = indices[:train_split]
16 val_idx = indices[train_split:val_split]
17 test_idx = indices[val_split:]
```

Listing 7: Train, Validation and Test sets

Each subset serves a defined purpose in the experimental pipeline. The **Training set** is utilized by the neural networks to learn the underlying patterns and update their weights. The **Validation set** is evaluated at the end of each training epoch; it acts as an unseen proxy to monitor the models' generalization ability and is crucial for preventing Overfitting during the training phase. Finally, the **Test set** remains completely locked and unobserved throughout the entire training and validation process, providing the final, unbiased evaluation of the models' real-world performance.

4.2 Hyperparameter Configuration

For the optimization of the network weights, the **Adam** (Adaptive Moment Estimation) optimizer was utilized, configured with a standard learning rate of 0.001. Adam is widely adopted in deep learning literature due to its computational efficiency and its ability to dynamically adapt the learning rate during training.

The **Mean Squared Error (MSE)** was selected as the Loss Function. MSE is ideal for this task because it squares the differences between the predicted and the actual target values, thereby heavily penalizing larger prediction errors.

Regarding the training loop, the models were configured to train for a maximum of 100 **epochs**. The **batch size** was set to 64, a value that provides an optimal balance between computational efficiency on the GPU and the stability of the gradient updates.

4.3 Early Stopping Mechanism

Early Stopping is a mechanism that monitors the validation loss (`val_loss`) at the end of each epoch and interrupts the training process if the performance stops improving. It was implemented using the Keras Callbacks API.

Specifically, the callback was configured with the following parameters:

- **Patience:** Set to 5, meaning the training will stop if the validation loss shows no improvement for 5 consecutive epochs.
- **Minimum Epochs:** To comply with the experimental requirements, the monitoring was configured to begin only after the 30th epoch (`start_from_epoch=30`). This ensures that the models undergo a sufficient initial training phase of at least 30 epochs before any early termination can occur.
- **Restore Best Weights:** Upon triggering the early stop, the network is automatically reverted to the specific weights of the epoch that achieved the lowest validation loss, rather than keeping the weights of the final (sub-optimal) step.

The Code Implementation

```
1 from tensorflow.keras.callbacks import EarlyStopping
2
3 # Define Early Stopping callback
4 # monitor='val_loss': We watch the validation error
5 # patience=5: Stop if it doesn't improve for 5 epochs
6 # start_from_epoch=30: Wait until epoch 30 before starting to
  monitor (Satisfies the >= 30 epochs rule)
7 # restore_best_weights=True: When it stops, keep the weights of the
  best epoch, not the last one
8 early_stop = EarlyStopping(monitor='val_loss', patience=5,
  start_from_epoch=30, restore_best_weights=True)
```

Listing 8: The Early Stopping Mechanism

4.4 Evaluation Metrics

Denormalization was required to revert the predictions back to their original physical scale. The predictions were multiplied by the appliance's maximum consumption value `max_app`. This critical step ensures that all calculated errors represent actual real-world electrical consumption in Watts.

The Code Implementation

```
1  # =====
2  # 2. Denormalization (Multiply by max_app)
3  # =====
4  # We remove the last dimension using np.squeeze so shape goes from (
   samples, seq_length, 1) to (samples, seq_length)
5  y_test_denorm = np.squeeze(y_test) * max_app
6  test_pred_rnn_denorm = np.squeeze(test_pred_rnn) * max_app
7  test_pred_lstm_denorm = np.squeeze(test_pred_lstm) * max_app
8  test_pred_gru_denorm = np.squeeze(test_pred_gru) * max_app
```

Listing 9: The Denormalization step

The Metrics

1. **Mean Absolute Error (MAE):** This metric measures the average magnitude of the errors in a set of predictions, without considering their direction. It is a linear score which means that all the individual differences are weighted equally in the average.

$$MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i| \quad (1)$$

2. **Root Mean Squared Error (RMSE):** This metric calculates the square root of the average of squared differences between prediction and actual observation. Because it squares the errors before averaging, RMSE heavily penalizes larger errors, making it highly sensitive to outliers (e.g., when the model completely misses a power spike).

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2} \quad (2)$$

3. **Maximum Error (Max Error):** This metric captures the worst-case scenario within a sequence. It is the absolute maximum residual error between the predicted and true power values, identifying the single minute where the model failed the most.

$$MaxError = \max_{i=1}^m (|y_i - \hat{y}_i|) \quad (3)$$

Where y_i is the actual (ground truth) consumption and $\hat{y}_i$ is the predicted consumption at timestep i .

5 Experimental Results

This section presents the findings from the experimental evaluation of the three Recurrent architectures. The analysis is divided into three parts: the training convergence behavior, the error metrics evaluation, and the visual assessment of the disaggregation performance on specific test sequences.

5.1 Training Convergence

The training process for all three models was successfully monitored and regulated by the early stopping mechanism. As observed in the training history plots (Figures 9, 10, and 11), the training loss dropped rapidly during the first few epochs, demonstrating the efficiency of the Adam optimizer.

The training process for all models was monitored using the Mean Squared Error (MSE) on both the training and validation sets. An early stopping mechanism was active to halt training if the validation loss did not improve for 5 consecutive epochs after epoch 30.

The RNN Model



Figure 9: Training and validation loss (MSE) over epochs for the Simple RNN model.

As observed in Figure 9, the Simple RNN model demonstrates a rapid decrease in loss during the initial epochs. The validation loss remains stable and closely follows the training loss, indicating a smooth convergence without signs of overfitting. The training naturally halted shortly after the 30-epoch threshold.

The LSTM Model

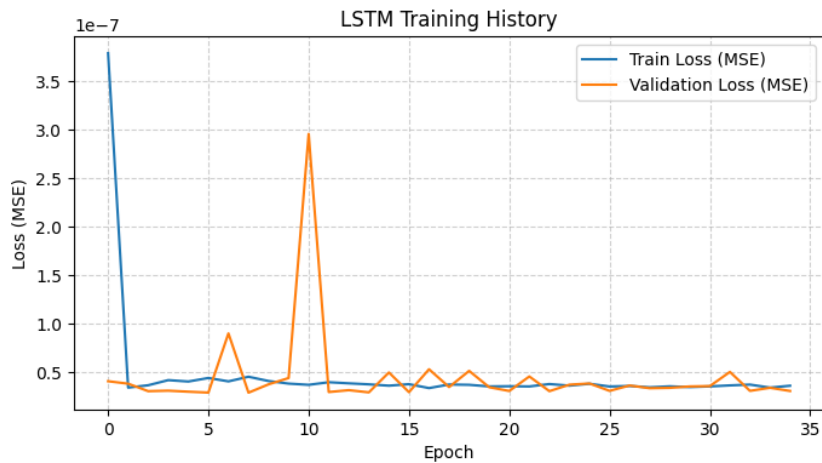


Figure 10: Training and validation loss (MSE) over epochs for the LSTM model. Notice the large validation spike around epoch 10.

Figure 10 presents the training history of the LSTM model. While the initial convergence is fast, there is a notable spike in the validation loss around epoch 10. Despite this temporary instability, the model recovers and stabilizes its error rate before the early stopping mechanism terminates the process.

The GRU Model

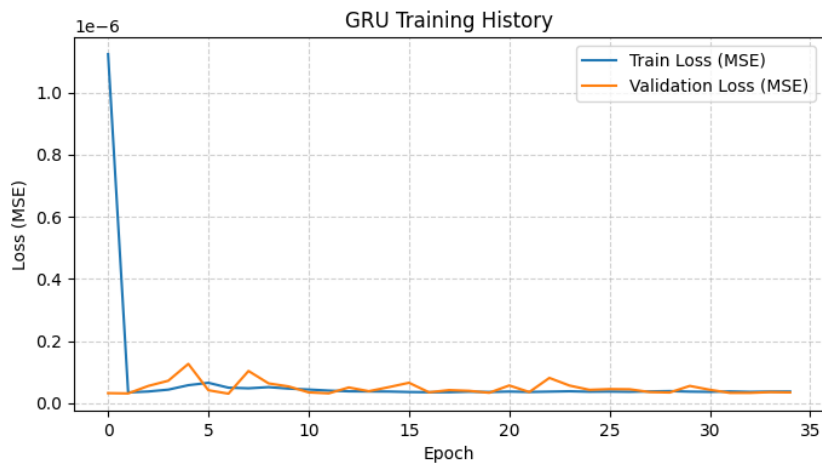


Figure 11: Training and validation loss (MSE) over epochs for the GRU model.

Figure 11 displays the training behavior of the GRU model. Similar to the Simple RNN, the GRU exhibits a smooth and stable convergence. The validation loss shows minor fluctuations but remains consistently low, successfully converging to its optimal weights.

5.2 Error Metrics Analysis

To evaluate the predictive accuracy of the models, the Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Maximum Error were calculated for each individual sequence in the denormalized test set.

RMSE per Sequence

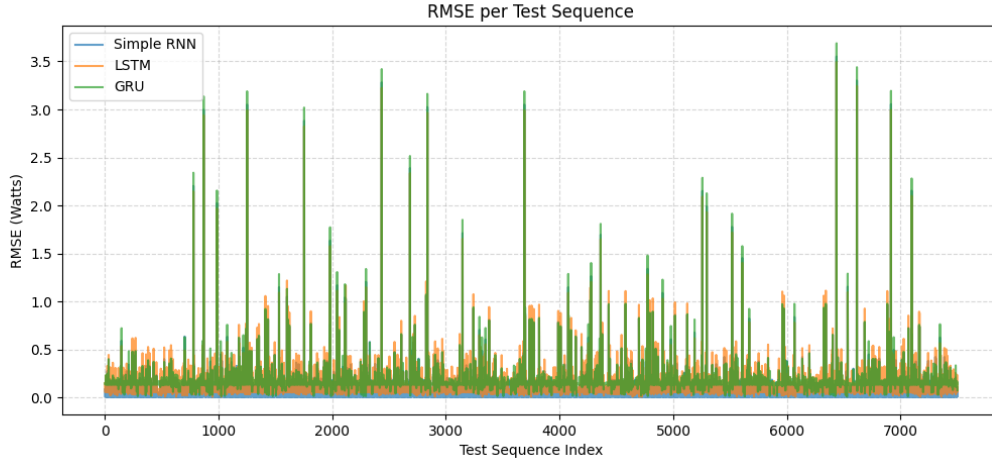


Figure 12: RMSE calculated per sequence in the test set.

Figure 12 illustrates the RMSE across all test sequences. Since RMSE squares the errors, it highlights larger deviations. The Simple RNN (blue line) maintains an error close to zero for the majority of the sequences. In contrast, the GRU (green line) shows frequent and large error spikes, while the LSTM (orange line) presents a continuous baseline error.

MAE per Sequence

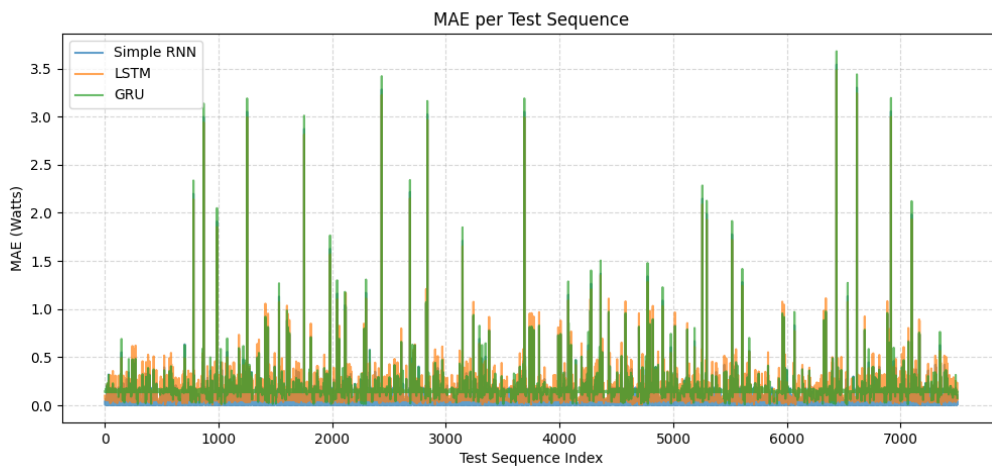


Figure 13: MAE calculated per sequence in the test set.

Figure 13 shows the MAE results, which represent the average magnitude of the prediction errors. The trend remains identical to the RMSE. The Simple RNN consistently outperforms the other two architectures, keeping the absolute error at minimal levels, whereas the LSTM and GRU fail to accurately approximate the ground truth.

Max Error per Sequence

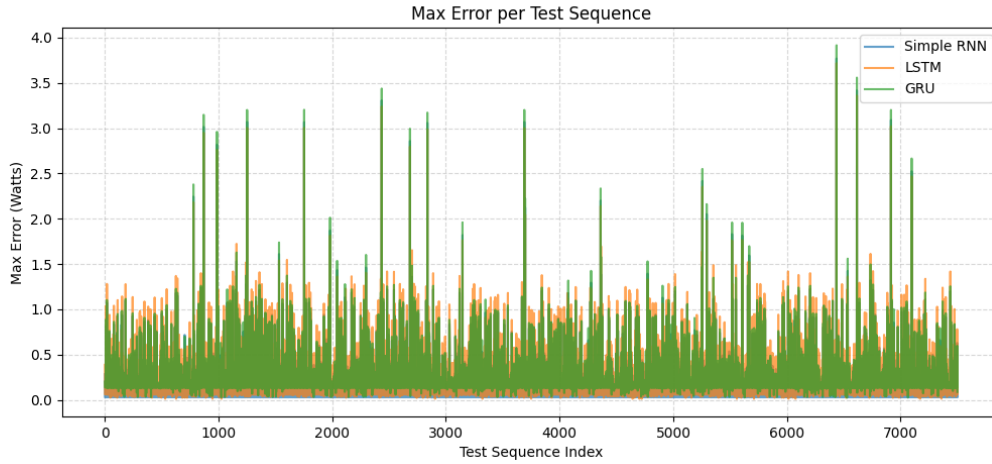


Figure 14: Maximum Error calculated per sequence in the test set.

Figure 14 highlights the maximum absolute error per sequence, effectively capturing the worst single-minute prediction within each time window. The GRU model exhibits severe deviations, missing the actual consumption by up to 4 Watts in certain sequences. The Simple RNN demonstrates high reliability, showing the lowest maximum errors across the dataset.

5.3 Visual Performance

To better understand the error metrics, four random test sequences were visually inspected. Two sequences represent periods where the appliance was turned OFF, and two represent periods where it was turned ON.

Test Sequence 3399

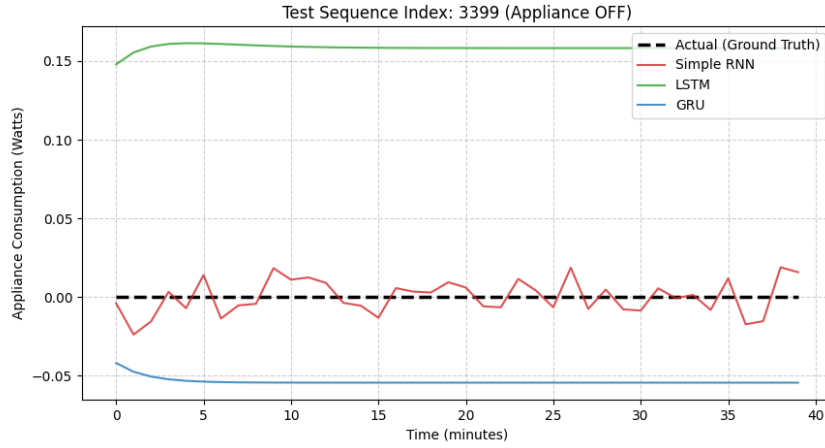


Figure 15: Appliance OFF state (Sequence 3399). Comparison of model predictions against the ground truth.

As shown in Figure 15, the actual consumption (ground truth) is zero. The Simple RNN correctly predicts values very close to zero. In contrast, the LSTM predicts a constant positive offset (approximately 0.16 Watts), and the GRU predicts a constant negative offset (approximately -0.05 Watts).

Test Sequence 3028

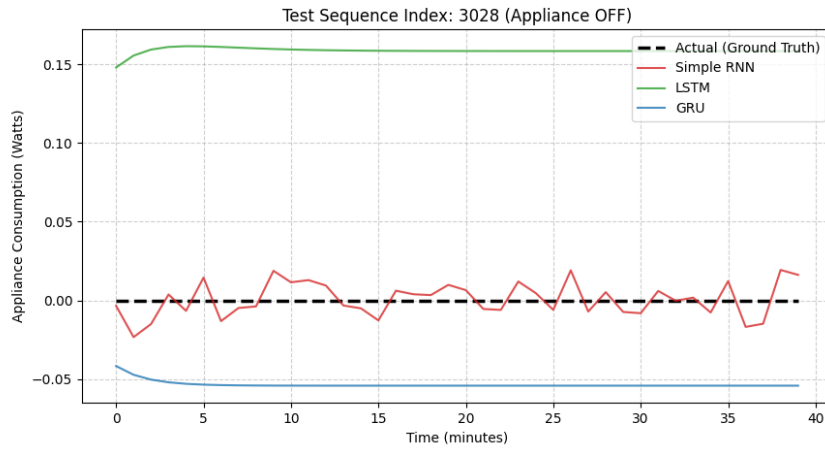


Figure 16: Appliance OFF state (Sequence 3028). Comparison of model predictions against the ground truth.

Figure 16 confirms the pattern observed in the previous OFF sequence. The Simple RNN continues to closely follow the zero line. The LSTM and GRU models maintain their respective flat predictions, failing to output a true zero value.

Test Sequence 316

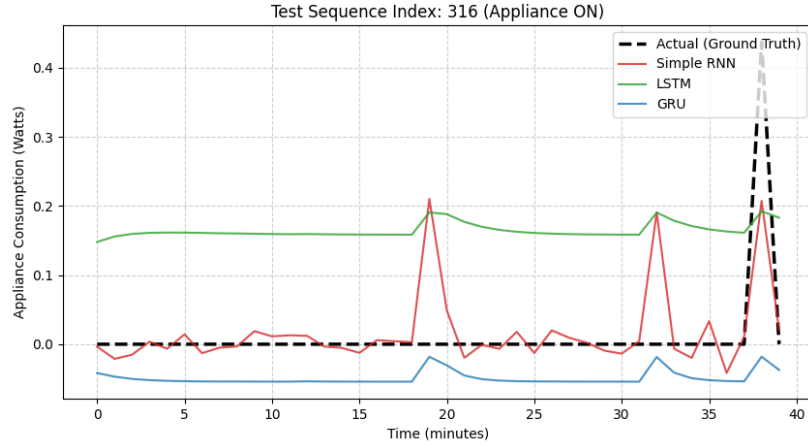


Figure 17: Appliance ON state (Sequence 316). The ground truth shows a sudden power spike.

Figure 17 illustrates an active state where the coffee machine draws a sudden, high amount of power. Both the LSTM and GRU models do not detect this activation, continuing to output their constant baseline values. The Simple RNN is the only model that successfully identifies and tracks the power spike.

Test Sequence 1352

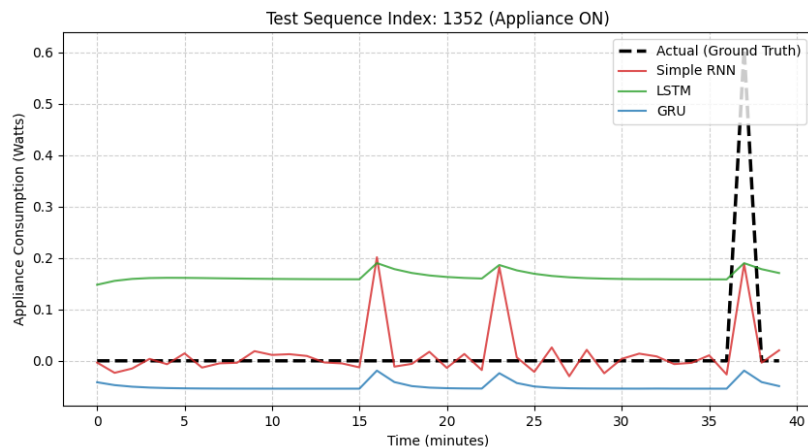


Figure 18: Appliance ON state (Sequence 1352). The Simple RNN successfully detects the activation.

Figure 18 provides another example of an ON state. Consistent with previous observa-

tions, the Simple RNN responds to the sudden changes in the aggregated signal. Meanwhile, the LSTM and GRU suffer from mode collapse, predicting a continuous average value regardless of the appliance’s actual state.

Performance Summary

The visual analysis explains the error rates of the advanced architectures. Because the coffee machine is turned OFF for the vast majority of the dataset, the LSTM and GRU models learned to predict a constant near-zero mean value to minimize their overall training loss, completely missing the rare ON events. The Simple RNN, having a shorter memory span, reacted more accurately to immediate input changes, making it the most suitable model for this specific appliance disaggregation task.

6 Conclusions

Purpose of this project

This project evaluated the performance of three fundamental recurrent neural network architectures—Simple RNN, LSTM, and GRU—applied to the task of energy disaggregation (NILM). The objective was to isolate the power consumption of a specific appliance (a coffee machine) from the aggregated household load using a Sequence-to-Sequence regression approach.

Results

The experimental results demonstrated that the **Simple RNN** was the most effective model for this specific task, consistently achieving the lowest error rates across all evaluated metrics (RMSE, MAE, and Max Error). While LSTM and GRU networks are generally considered superior for sequence modeling due to their advanced gating mechanisms, they suffered from *mode collapse* in this scenario.

Because the coffee machine remains inactive (OFF) for the vast majority of the time, the advanced architectures over-smoothed their predictions into a constant baseline, failing to detect the abrupt power spikes. In contrast, the shorter memory span of the Simple RNN allowed it to react accurately to sudden activation events.

Limitations and Further Improvements

Despite the clear results, the current methodology has certain limitations. The evaluation was restricted to a simple ON/OFF appliance with brief, high-power activation windows.

For complex, multi-state appliances with long-duration operational cycles (such as washing machines or dishwashers), the long-term memory capabilities of LSTMs and GRUs might prove necessary and could potentially outperform a Simple RNN.

Future work could explore the application of Bidirectional architectures (e.g., BiLSTM or BiGRU), which analyze sequences in both forward and backward directions to provide better temporal context.

Additionally, implementing automated hyperparameter optimization techniques (such as Grid Search or Random Search) could help the advanced models overcome mode collapse, improving their disaggregation performance across a wider variety of household appliances.

References

- [1] G. W. Hart, “Nonintrusive appliance load monitoring | IEEE journals & magazine | IEEE xplore,” *Proceedings of the IEEE*, vol. 80, no. 12, pp. 1870–1891, 1992. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/192069>
- [2] H. Salehinejad, S. Sankar, J. Barfett, E. Colak, and S. Valaee, “Recent advances in recurrent neural networks.” [Online]. Available: <http://arxiv.org/abs/1801.01078>
- [3] Z. C. Lipton, J. Berkowitz, and C. Elkan, “A critical review of recurrent neural networks for sequence learning.” [Online]. Available: <http://arxiv.org/abs/1506.00019>
- [4] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [5] Y. Yu, X. Si, C. Hu, and J. Zhang, “A review of recurrent neural networks: LSTM cells and network architectures,” *Neural Computation*, vol. 31, no. 7, pp. 1235–1270, 2019. [Online]. Available: https://doi.org/10.1162/neco_a_01199
- [6] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2014, pp. 1724–1734. [Online]. Available: <http://aclweb.org/anthology/D14-1179>
- [7] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” 2014. [Online]. Available: <http://arxiv.org/abs/1412.3555>
- [8] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Advances in Neural Information Processing Systems*, vol. 27. Curran Associates, Inc., 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/hash/5a18e133cbf9f257297f410bb7eca942-Abstract.html
- [9] J. Kelly and W. Knottenbelt, “Neural NILM: Deep neural networks applied to energy disaggregation,” in *Proceedings of the 2nd ACM International Conference on Embedded Systems for Energy-Efficient Built Environments*, ser. BuildSys ’15. Association for Computing Machinery, 2015, pp. 55–64. [Online]. Available: <https://dl.acm.org/doi/10.1145/2821650.2821672>